

Lab 1 Report Sections: Task 10 and Task 11

William Lebrato, aile20

1 Task 10: Parallel Gaussian Elimination

1.1 Implementation

Gaussian elimination has two main steps for each pivot row k :

1. **Division Step:** Normalize the pivot row by dividing everything by $A[k][k]$.
2. **Elimination Step:** For all rows $i > k$, zero out the element in column k using:

$$A[i][j] = A[i][j] - A[i][k] \cdot A[k][j].$$

We can only parallelize the elimination step because the pivot row needs to be finished before other rows can use it. To do this, we used a **cyclic row distribution**. For a specific pivot k , thread t works on these rows:

$$i = k + 1 + t, k + 1 + t + \text{NUM_THREADS}, \dots$$

This spreads the work evenly without needing complicated load-balancing code. Note that if the number of rows left isn't perfectly divisible by the thread count, the **last few threads get slightly fewer rows**. This is just a natural result of the division.

For every pivot, we spawn 16 threads. Each thread gets:

- its thread ID,
- the pivot index k ,
- and pointers to the shared matrix and vectors.

After launching the threads, the main thread simply waits using `pthread_join()` to make sure all elimination work is done before starting the next pivot.

1.2 Execution Results

We tested this on a 2048×2048 matrix.

Table 1: Execution times and speedup for Parallel Gaussian Elimination

Configuration	Time (s)	Speedup
Sequential	12.592	1.00
16 Threads	4.205	2.99

The resulting speedup is:

$$S = \frac{12.592}{4.205} \approx 2.99 \times .$$

1.3 Analysis

The parallel version is nearly $3\times$ faster. The biggest slowdown comes from creating and joining threads for every single pivot iteration. Even so, the elimination workload is heavy enough that the parallelism is worth the synchronization cost. The cyclic distribution also kept the load balanced, with only the final threads occasionally getting less work.

2 Task 11: Parallel Quicksort

2.1 Implementation

Quicksort uses recursion, but creating a new thread for every recursive call is too expensive and would create way too many threads. To fix this, we used a **fixed-size thread pool** and a **shared task queue**.

Thread Pool

We create a set number of worker threads (`NUM_THREADS`) at the start. Each worker runs a loop where it:

1. waits for a task,
2. dequeues the next task (protected by a mutex),
3. runs `quick_sort()` on it,
4. signals that it is done and loops back to wait again.

This prevents the huge overhead of making threads dynamically inside the recursion.

Task Queue Structure

We used a FIFO queue built as a linked list. Each task struct has a pointer

```
struct QuickSortTask* next;
```

to link to the next one. The queue tracks both the **head** and **tail**:

- If the queue is empty, the new task is both the head and the tail.
- Otherwise, we just add the new task to the tail in $\mathcal{O}(1)$ time.

Workers take tasks from the **head**. Since adding a task wakes up exactly one sleeping worker with `pthread_cond_signal()`, tasks usually get picked up instantly, so the queue rarely gets long.

Parallelization Strategy

In the recursive `quick_sort()` function:

- After partitioning, we put the **left subproblem** into the queue as a new task, but only if the recursion depth is less than `MAX_DEPTH`.
- The **right subproblem** is handled immediately by the current thread.

This keeps the workers busy and cuts down on overhead. If we hit `MAX_DEPTH`, we process both sides sequentially so we don't waste time creating tasks for tiny subarrays.

2.2 Execution Results

We tested the Quicksort on an array size of 64×2^{20} (about 67 million integers).

Table 2: Execution times and speedup for Parallel Quicksort

Configuration	Avg Time (s)	Speedup
Sequential	16.36	1.00
4 Threads (Depth 2)	10.97	1.49
8 Threads (Depth 3)	10.37	1.58
16 Threads (Depth 4)	7.36	2.22
64 Threads (Depth 12)	3.71	4.41
512 Threads (Depth 15)	3.57	4.58

2.3 Analysis

Using a thread pool definitely improved performance. We hit a speedup of $4.41\times$ with 64 threads, passing the $4\times$ mark. Going up to 512 threads only gave a small boost (3.71s to 3.57s), which suggests that memory bandwidth and synchronization costs are limiting further speedup.

The FIFO task queue balances the load well because workers grab tasks as soon as they are made. Since the condition variable wakes threads up immediately, the queue stays short. The linked-list structure makes adding and removing tasks very fast ($\mathcal{O}(1)$) even with many threads.

Overall, the speedup is good but not linear. The main limits are:

- shared-memory bandwidth,
- threads fighting for the mutex on the task queue,
- and less work per task at deeper recursion levels.